Training a physics-informed neural network with a fixed quantum layer on a superconducting quantum processor

Gubio G. de Lima[1,*] and Erick [TODO: sobrenome][2]
[1] *[TODO: afiliação (CBPF / UFSCar)]*
[2] *[TODO: afiliação]*

[TODO: Resumo – a escrever por último.]

I. INTRODUCTION

[TODO: Introdução – a escrever.]

II. THEORETICAL FRAMEWORK

A. Benchmark problem

We solve the viscous Burgers equation on the periodic unit interval,

$$\partial_t u + u\,\partial_x u = \nu\,\partial_x^2 u, \qquad x \in [0,1),\ t \in [0,T], \tag{1}$$

with $u(x+1,t) = u(x,t)$, $u(x,0) = \sin(2\pi x)$ and $\nu = 0.01$. Equation (1) is the simplest partial differential equation (PDE) that combines nonlinear advection with diffusion [1]: for $\nu = 0.01$ the initial sine steepens into a front at $x = 1/2$, which is already sharp at $T = 0.1$; at $T = 1$ the wave has been strongly damped ($\|u\|_{\mathrm{rms}} \simeq 0.21$) and the problem is harder for the networks because the solution has to be propagated over a ten times longer horizon. We consider the *forward* problem: ν is known and no solution data enter the training. A reference solution u_{FDM} is computed by explicit finite differences (Appendix A) and is used only for evaluation. The accuracy of a model is measured on $u(x,T)$ at 200 equispaced points by the mean squared error (MSE) and the relative L_2 error $\epsilon_{L_2} = \|u - u_{\mathrm{FDM}}\|_2 / \|u_{\mathrm{FDM}}\|_2$. A model that does not evolve the initial condition, $u(x,T) = \sin 2\pi x$, has MSE $= 3.93 \times 10^{-2}$ at $T = 0.1$ and 2.82×10^{-1} at $T = 1$; we refer to this as the *no-evolution level*.

B. Physics-informed neural networks

A physics-informed neural network (PINN) [2, 3] represents the solution by a parametrized function $u_\Theta(x,t)$ and determines the parameters Θ by minimizing the residual of the PDE at a set of collocation points $\{(x_i,t_i)\}_{i=1}^{N_{\mathrm{col}}}$,

$$\mathcal{L}(\Theta) = \frac{1}{N_{\mathrm{col}}} \sum_{i=1}^{N_{\mathrm{col}}} r_i^2, \tag{2}$$

with $r_i = \big(\partial_t u_\Theta + u_\Theta\,\partial_x u_\Theta - \nu\,\partial_x^2 u_\Theta\big)(x_i,t_i)$. Boundary and initial conditions are usually added as penalty terms; here both are imposed exactly by construction [4]. The network acts on the periodic feature vector $\boldsymbol{\xi}(x,t) = (\sin 2\pi x, \cos 2\pi x, t/T)$ and its output enters

$$u_\Theta(x,t) = \sin(2\pi x) + \frac{t}{T}\,\big(\mathcal{N}_{\mathrm{post}} \circ \mathcal{M} \circ \mathcal{N}_{\mathrm{pre}}\big)\big(\boldsymbol{\xi}(x,t)\big), \tag{3}$$

so that u_Θ is exactly 1-periodic in x and equals the initial condition at $t = 0$, and Eq. (2) is the complete loss. The classical pre-network $\mathcal{N}_{\mathrm{pre}}(\cdot;\boldsymbol{\phi}) : \mathbb{R}^3 \to \mathbb{R}^n$ (one hidden layer of 10 tanh units, linear output) maps the features to a latent vector $\boldsymbol{\theta} \in \mathbb{R}^n$, a middle block $\mathcal{M} : \mathbb{R}^n \to \mathbb{R}^n$ transforms it into $\boldsymbol{y}$, and the classical post-network $\mathcal{N}_{\mathrm{post}}(\cdot;\boldsymbol{\psi}) : \mathbb{R}^n \to \mathbb{R}$ (one hidden layer of 10 tanh units) produces the correction to the initial profile [Fig. 1(a)]. The width n of the latent space is the only size parameter of the model. The classical PINN and the hybrid quantum PINN (HQPINN) differ *only* in $\mathcal{M}$, so that any difference in their behaviour can be attributed to the middle block.

The derivatives in r_i are computed by second-order central differences of the network itself on the five-point stencil $\{(x_i,t_i), (x_i \pm h, t_i), (x_i, t_i \pm \tau)\}$ with $h = 1/512$ and $\tau = 10^{-3}$ [Fig. 1(b)], rather than by automatic differentiation with respect to the inputs. This choice is dictated by the hardware: a quantum processor returns function values, not derivatives with respect to (x,t), and all stencil nodes of a batch can be evaluated in a single pass. The truncation error of the stencil is negligible at the accuracy reached here (Appendix D). At every optimizer step a fresh set of N_{col} collocation points is drawn uniformly in $[0,1) \times [\tau, T-\tau]$.

In the *classical PINN* the middle block is a trainable network,

$$\mathcal{M}_{\mathrm{C}}(\boldsymbol{\theta}) = \mathcal{N}_{\mathrm{mid}}(\boldsymbol{\theta};\boldsymbol{\chi}), \tag{4}$$

with one hidden tanh layer, so that the classical model is a tanh network $3 \to 10 \to n \to m \to n \to 10 \to 1$ with a bottleneck of width n; the hidden width is $m = n$ in the long-budget simulations and $m = 10$ in the hardware-budget control (Table V; the two coincide at $n = 10$, 491 parameters).

* gubio.gomes@estudante.ufscar.br

C. Hybrid quantum PINN with a fixed quantum layer

In the HQPINN the latent vector is the set of rotation angles of an n-qubit circuit,

$$U(\boldsymbol{\theta}) = W_{\text{brick}} \bigotimes_{q=1}^{n} R_Y(\theta_q), \qquad R_Y(\theta) = e^{-i\theta Y/2}, \quad (5)$$

where W_{brick} is one brickwork layer of nearest-neighbour CNOT gates, first on the pairs $(1,2),(3,4),\dots$ and then on $(2,3),(4,5),\dots$ ($n-1$ gates in total). The circuit acts on $|0\rangle^{\otimes n}$ and the layer output is the vector of local magnetizations,

$$y_q(\boldsymbol{\theta}) = \mathrm{Tr}\big[Z_q\, \mathcal{E}_{\boldsymbol{\theta}}\big(|0\rangle\langle 0|^{\otimes n}\big)\big], \qquad q = 1,\dots,n, \quad (6)$$

where $\mathcal{E}_{\boldsymbol{\theta}}$ is the channel actually implemented by the device. On hardware y_q is estimated from S shots, $\hat{y}_q = y_q + \xi_q$ with $\mathbb{E}[\xi_q] = 0$ and $\mathrm{Var}[\xi_q] = (1-y_q^2)/S \le 1/S$.

The defining feature of the model is that *the circuit has no free parameter*: all trainable parameters belong to the classical networks, and the quantum layer is a fixed nonlinear map $\boldsymbol{\theta} \mapsto \boldsymbol{y}$. The HQPINN has $21n + 61$ trainable parameters (271 for $n = 10$), roughly half of the classical model of the same width. This is the opposite design choice to the variational hybrid PINNs studied so far [5–9], and connects the HQPINN to fixed-substrate approaches such as quantum feature maps [10], quantum reservoir computing and extreme learning machines [11, 12] and quanvolutional networks [13], none of which has been used to minimize a PDE residual.

a. Function class. The structure of the quantum layer is captured by the following statement (proof in Appendix B).

Proposition 1 *Let $\mathcal{C}_q \subseteq \{1,\dots,n\}$ be the backward light cone of Z_q in the circuit (5). For any noise model in which every gate is followed by a $\boldsymbol{\theta}$-independent, trace-preserving channel acting on the qubits of that gate (readout errors included), y_q depends only on $\{\theta_j\}_{j\in\mathcal{C}_q}$ and is a multilinear trigonometric polynomial of degree one in each of them,*

$$y_q(\boldsymbol{\theta}) = \sum_{\boldsymbol{\alpha}\in\{0,c,s\}^{|\mathcal{C}_q|}} c_{\boldsymbol{\alpha}}^{(q)} \prod_{j\in\mathcal{C}_q} g_{\alpha_j}(\theta_j), \quad (7)$$

with $g_0 = 1$, $g_c = \cos$, $g_s = \sin$, and device-dependent real coefficients $c_{\boldsymbol{\alpha}}^{(q)}$. For one brickwork layer $|\mathcal{C}_q| \le 4$, i.e., at most $3^4 = 81$ coefficients per output.

The hypothesis holds on IBM devices because $R_Y(\theta)$ is compiled into fixed microwave pulses and a *virtual* R_Z rotation, so the physical operations, and hence their errors, do not depend on θ. Equation (7) has three consequences that guide both the theory and the implementation. (i) The quantum layer injects a finite Fourier structure into the latent representation [14]: the pre-network learns where to place the angles, and the circuit multiplies harmonics of different latent coordinates, $\cos\theta_a\cos\theta_b = \frac{1}{2}[\cos(\theta_a+\theta_b)+\cos(\theta_a-\theta_b)]$, in the spirit of Fourier-feature networks [15, 16]. (ii) Noise does not change the function class, only the coefficients; this is what makes a calibrated surrogate of the device possible (Sec. II D). (iii) The quantum layer has no trainable parameter, so the barren-plateau phenomenon of variational circuits [17, 18] does not arise; what remains is the possible concentration of the map $\boldsymbol{\theta} \mapsto \boldsymbol{y}$ itself, which is controlled by the size of the light cones (Remark 1).

Remark 1 *A CNOT network acts on the computational basis as a linear map over* GF(2), $\boldsymbol{b} \mapsto M\boldsymbol{b}$. *For the noiseless circuit (5) the Heisenberg evolution of Z_q is the Pauli string $\prod_{j\in\mathcal{S}_q} Z_j$ with $\mathcal{S}_q = \{j : M_{qj} = 1\} \subseteq \mathcal{C}_q$, and*

$$y_q(\boldsymbol{\theta}) = \prod_{j\in\mathcal{S}_q} \cos\theta_j, \qquad |\mathcal{S}_q| \le 3. \quad (8)$$

The ideal layer is therefore efficiently classically computable: the noiseless HQPINN is a classical network with cosine-product activations. Its simulation results (Sec. IV A) measure the merit of this architecture, *not a quantum advantage, in line with the general results on classical surrogates of quantum learning models [19]. For angles spread over a full period,* $\mathrm{Var}[y_q] = 2^{-|\mathcal{S}_q|}$, *so deeper entangling layers trade expressivity (longer products) against concentration.*

D. Training with the quantum processor in the loop

Training the HQPINN on a quantum processor raises two problems. First, the device returns $\hat{\boldsymbol{y}}$ but not $\partial\boldsymbol{y}/\partial\boldsymbol{\theta}$. Parameter-shift rules [20–22] give unbiased derivatives at the price of $2n$ extra circuits per evaluation point, and SPSA [23, 24] gives a single random directional derivative per step. Second, the residual (2) is built from finite differences: if the five stencil nodes were measured independently, the shot noise σ_u of u would enter $\partial_x^2 u$ amplified by $\sqrt{6}/h^2 \approx 6.4\times10^5$ (Appendix D), which would bury the physical signal. Both problems are solved by Proposition 1, through a scheme analogous to the physics-aware training of physical neural networks [25], in which the physical system computes the forward pass and a digital model of it provides the backward pass [Fig. 1(c)]:

1. *Calibration.* Before training, one job evaluates the circuit at random angles $\theta_j \sim \mathcal{U}(-\pi,\pi)$ on every n-qubit chain of the chip in parallel. The coefficients of Eq. (7) are obtained by linear least squares, which yields a classical surrogate $\hat{f}(\boldsymbol{\theta})$ of the device with its average noise built in [26].

2. *Training.* At each epoch the device is measured only at the N_{col} stencil centres $\boldsymbol{\theta}_c$. Every node $\boldsymbol{\theta}$ of the stencil centred at c receives

$$\tilde{\boldsymbol{y}}(\boldsymbol{\theta}) = \hat{f}(\boldsymbol{\theta}) + \mathrm{sg}\big[\hat{\boldsymbol{y}}^{\,\mathrm{QPU}}(\boldsymbol{\theta}_c) - \hat{f}(\boldsymbol{\theta}_c)\big], \quad (9)$$

where sg stops the gradient (straight-through estimator [27]). At the centre $\tilde{\boldsymbol{y}} = \hat{\boldsymbol{y}}^{\mathrm{QPU}}$, so the value of u entering the nonlinear term $u\,\partial_x u$ is the measured one; across the stencil the variation of $\tilde{\boldsymbol{y}}$ is that of the smooth surrogate, so the shot noise is not amplified by the finite differences; and the gradient is $\partial\tilde{\boldsymbol{y}}/\partial\boldsymbol{\theta} = \partial\hat{f}/\partial\boldsymbol{\theta}$, available analytically. Every K_{fit} epochs the surrogate is refitted with the calibration data and all measurements accumulated so far, so that it tracks the device in the region of angles actually visited by the network.

3. *Validation.* A final job evaluates $u(x, T)$ with the trained network and the circuit run on the device.

A training run of K epochs therefore costs $K + 2$ jobs and N_{col} circuit evaluations per epoch, independently of the number of trainable parameters (Table III). Because the surrogate has the exact functional form of the device response, the method does not approximate the circuit; its only bias is the statistical error of the fitted coefficients and the part of the device behaviour that the time-independent, chip-averaged model (7) cannot capture (chain-to-chain variation, drift, crosstalk), which we quantify in Sec. IV C.

III. METHODS

The numerical and experimental protocols are summarized here; Appendices C–F contain the full algorithms, the implementation of the hardware loop and all hyperparameters. The results are organized in two groups: *simulations* (noiseless circuits, shot noise only, and the noise model of the device) and *real-QPU* executions.

A. Simulations

a. Long-budget, same-protocol sweep. Both models are trained with the same algorithm and hyperparameters (Algorithm 1): AdamW [28] with learning rate 10^{-3} reduced on plateaus, a fresh batch of N_{col} collocation points at every step, up to 2×10^4 steps, followed by at most 10^3 L-BFGS iterations [29] on the last batch. The residual is monitored on a fixed validation set of N_{col} random points, and training stops when it reaches the residual of the reference solution itself on the same points and with the same stencil: below this *FDM floor* the network would be fitting the discretization error of the reference. The best monitored snapshot is returned. We sweep $n = 2, \ldots, 15$, $N_{\mathrm{col}} \in \{10, 100\}$ and $T \in \{0.1, 1\}$, with 50 seeds per configuration and model (5600 runs), seed s fixing the initialization and the validation set. [**TODO:** arquivar no repositório o script que gerou a varredura (`Experiment1Modified`, protocolo `fixed_validation_v2`); a versão local de `run_qubit_sweep.py` monitora o lote de treino.] The ideal quantum layer is evaluated with the closed form (8), which is exact for the noiseless circuit and differentiable. A complementary sweep of the HQPINN alone covers N_{col} from 10 to 500 for $n \in \{3, 5, 7, 10, 15\}$ (Appendix G).

b. Hardware-budget simulations. To separate the effect of noise from that of the short training affordable on hardware, the HQPINN is also trained offline with exactly the protocol of the hardware run (110 epochs, $N_{\mathrm{col}} = 10$, same learning-rate schedule, same initialization and collocation stream per seed; Sec. III B) with (i) the ideal layer on all stencil nodes and (ii) the ideal layer plus binomial shot noise with $S = 1024$ at the stencil centres, propagated with Eq. (9) and a perfect surrogate. Seeds 0–4 are used.

c. Noisy emulation. The device is emulated with the `FakeBerlin` noise model of Qiskit [30], which reproduces the calibrated gate errors, T_1/T_2 relaxation, readout errors and coupling map of `ibm_berlin` but not crosstalk or temporal drift. Each 10-qubit chain is simulated exactly as a density matrix and $S = 1024$ shots are sampled. The emulated training uses the same physical chains, seed, collocation stream and protocol as the real run (Algorithm 2).

B. Real-QPU execution

The hardware experiment ran on `ibm_berlin`, an IBM Nighthawk processor with 120 transmon qubits on a square lattice and native CZ gates, on 8 October 2026, within a single dedicated runtime `Session`. The circuit (5) with $n = 10$ was laid out on 8 disjoint chains of 10 coupled qubits [Fig. 3(a)], selected from the current calibration data (12 candidate chains, 4 discarded for a high error score), and the 8 copies were transpiled into a single packed circuit of depth 12 with 72 CZ gates and no SWAPs. All expectation values were estimated with $S = 1024$ shots (precision 1/32) and *without* error suppression or mitigation (raw counts). The run consisted of one calibration job (128 parameter sets on 8 chains, 1024 samples per output), 110 training jobs (one per epoch, the 10 stencil centres distributed over the chains) and one validation job (200 points, each measured once on every chain and averaged), i.e., 112 jobs, 548 parameter sets and 5.6×10^5 shots in 54 min of wall-clock time. Training used AdamW with learning rate 0.1 decreased by 0.01 every 10 epochs down to 0.01, gradient-norm clipping at 1 and $K_{\mathrm{fit}} = 10$. The same settings were used in an earlier FakeBerlin batch of five seeds, and the hardware seed (seed 0) is the best of that batch. Device characteristics are listed in Table IV.

As controls with the *same* budget we (a) evaluated on `ibm_berlin` (one prediction job) the network trained on FakeBerlin, and (b) trained the classical PINN with exactly the hardware protocol for $n = 1, \ldots, 10$ and seeds 0–4 (50 runs). Since all randomness of the hardware training is in the measured values, which were recorded, the training is reproducible offline: feeding the recorded measurements into the same update reproduces

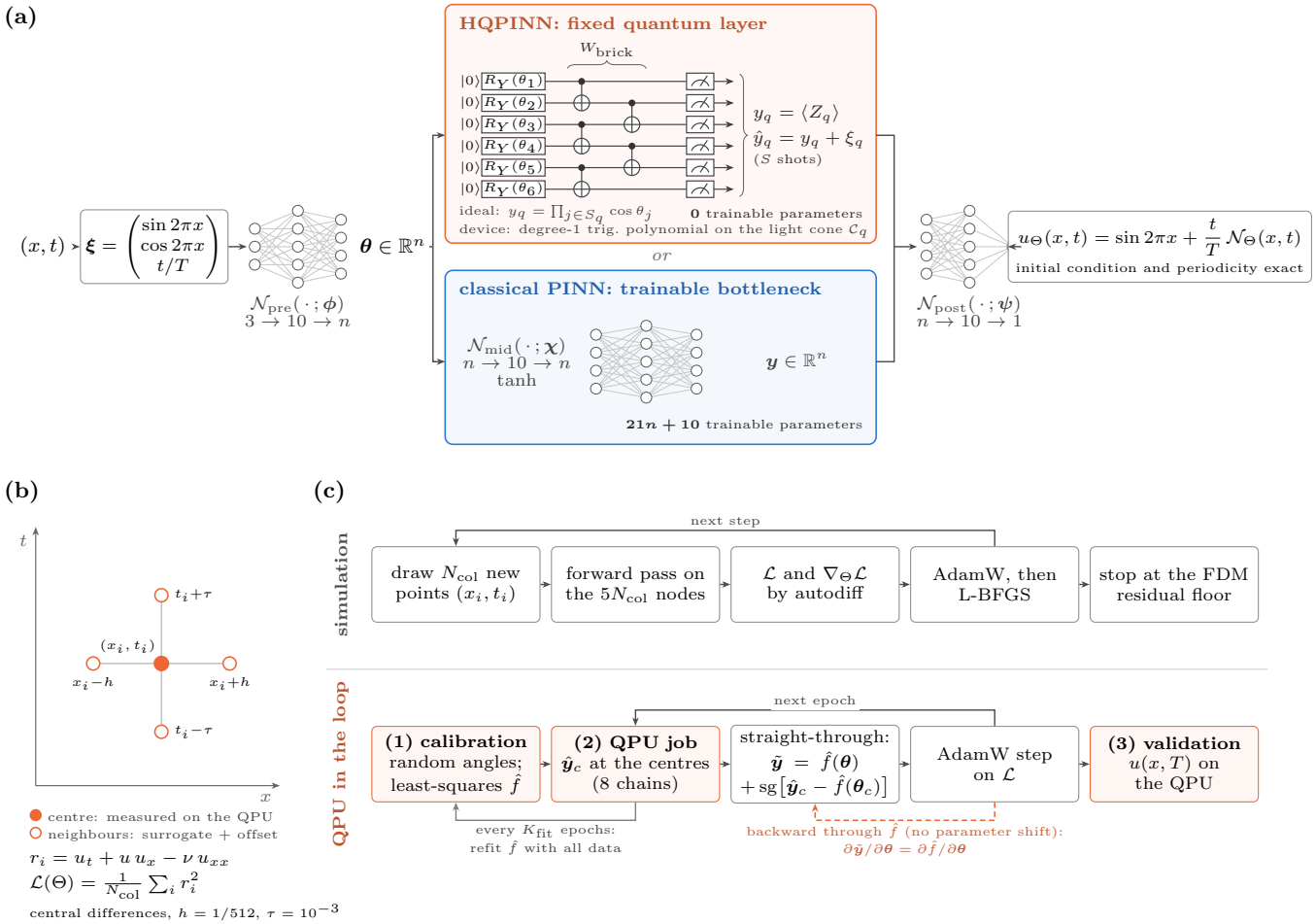


FIG. 1. Classical PINN and hybrid quantum PINN (HQPINN). (a) Architecture. Both models share the classical pre-network $\mathcal{N}_{\text{pre}}$, which maps the periodic features $\boldsymbol{\xi}$ to the latent vector $\boldsymbol{\theta} \in \mathbb{R}^n$, and the classical post-network $\mathcal{N}_{\text{post}}$; the initial condition and the periodicity are imposed exactly by Eq. (3). In the HQPINN (orange) the middle block is a *fixed* quantum circuit—one $R_Y(\theta_q)$ rotation per qubit followed by one brickwork layer of CNOTs—whose output is the vector of local expectation values $y_q = \langle Z_q \rangle$, estimated with S shots; the circuit has no trainable parameter. Six of the $n = 10$ qubits used on hardware are drawn. In the classical PINN (blue) the middle block is a trainable bottleneck $\mathcal{N}_{\text{mid}}$. (b) Physics-informed residual. The derivatives in r_i are central differences of the network on a five-point stencil; on hardware only the centre is measured, and the neighbours receive the surrogate value plus the same measured offset [Eq. (9)]. (c) Training workflows. In simulation the full stencil is evaluated exactly and the gradient is obtained by automatic differentiation. With the quantum processor in the loop, (1) one calibration job fits the trigonometric surrogate $\hat{f}$ of the device [Eq. (7)]; (2) at every epoch one job measures the circuit at the N_{col} stencil centres on all chains in parallel, the forward pass uses the measured values and the backward pass goes through $\hat{f}$ (straight-through estimator, dashed); $\hat{f}$ is refitted every K_{fit} epochs with all measurements; (3) one final job evaluates the trained network on the device.

the trained weights bit for bit (Appendix E). We use this replay to evaluate the network after every epoch.

IV. RESULTS

A. Simulations

a. Same protocol, long budget. Figure 2 and Table I compare the two models over 5600 runs. At $T = 0.1$ with $N_{\text{col}} = 100$ both models reach the same median MSE, $\simeq 7\times10^{-6}$ ($\epsilon_{L_2} \simeq 0.4\%$) for $n \geq 4$ [Fig. 2(a)]; this level is set by the reference solution and is reached by the HQPINN for every $N_{\text{col}} \geq 30$ (Appendix G). With only $N_{\text{col}} = 10$ points per step the median MSE of the HQPINN is lower than that of the classical PINN for every width, by a factor between 1.2 and 4.5, and its distribution over seeds is narrower [Fig. 2(b)]. Neither model has failed runs at $T = 0.1$ under this protocol (largest MSE 2.2×10^{-3} for the classical PINN and 1.2×10^{-3} for the HQPINN among 1400 runs each). The difference is more pronounced in convergence than in final accuracy: with $N_{\text{col}} = 100$, 98% of the HQPINN runs with $n \geq 5$ reach the physical target, against 55% of the classical ones, and for $n = 10$

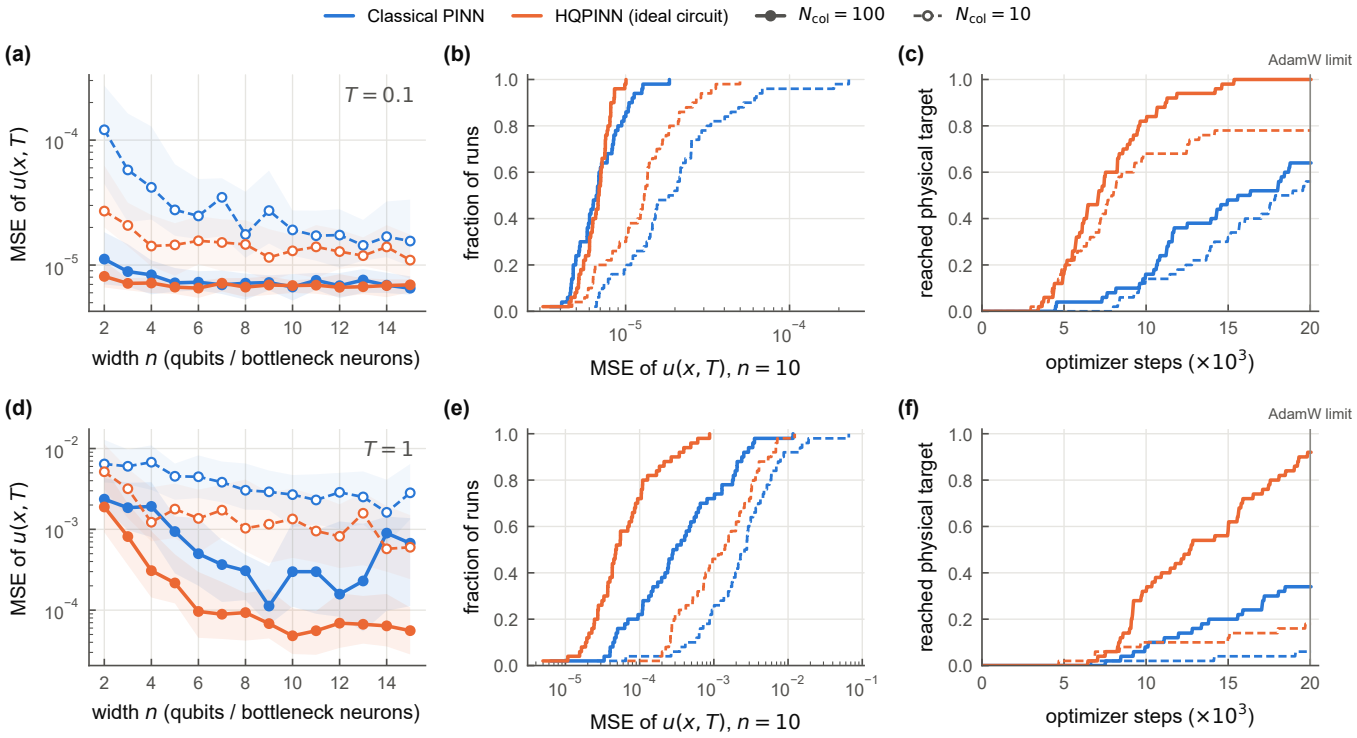


FIG. 2. Noiseless simulations, classical PINN (blue) and HQPINN with the ideal circuit (orange) trained with the same protocol; 50 seeds per point. Top row, $T = 0.1$; bottom row, $T = 1$. Solid lines, $N_{\text{col}} = 100$ collocation points per step; dashed lines with open markers, $N_{\text{col}} = 10$. (a,d) Median MSE of $u(x, T)$ versus the width n of the latent space (number of qubits of the HQPINN, bottleneck width of the classical PINN); bands, interquartile range. (b,e) Empirical distribution of the MSE over the 50 seeds for $n = 10$. (c,f) Fraction of the runs, $n = 10$, that have reached the physical stopping target (the residual of the reference solution, Sec. III A) as a function of the number of optimizer steps; the AdamW phase ends at 2×10^4 steps.

the median run stops after 7.1×10^3 steps against 1.6×10^4 [Fig. 2(c)].

At $T = 1$ the advantage of the HQPINN extends to the final accuracy [Figs. 2(d)–(f)]. Its median MSE is lower for every width and both values of N_{col}, by a factor of up to 14 for $N_{\text{col}} = 100$; for $n = 10$ it is 4.8×10^{-5} ($\epsilon_{L_2} = 3.3\%$) against 3.0×10^{-4} (8.2%), and 92% of the runs reach the target against 34%. The classical PINN improves non-monotonically with n, and its best median (1.1×10^{-4} at $n = 9$, 430 parameters) is above that of the HQPINN for every $n \geq 6$ (187 parameters and more) [Fig. 6(c)]. Since the ideal quantum layer is a fixed cosine-product map (Remark 1), these results show that a parameter-free, bounded and periodic middle block conditions the residual minimization better than a trainable bottleneck, in particular for the long horizon; they do not rely on any quantum effect. At the hardware budget [$T = 0.1$, 110 epochs, $n = 10$, seeds 0–4; Fig. 4], replacing tanh by sin in the classical middle block lowers the median ϵ_{L_2} from 31.1% to 20.9% (7.0–27.2%) under the learning-rate schedule of the HQPINN (9.1% for the noiseless HQPINN), and from 11.4% to 10.6% (8.7–12.5%) under the best schedule of the classical model (Sec. V): a periodic activation closes part of the gap at a common schedule and has little effect once the schedule is tuned. [**TODO:** repetir o controle com `seno=True` no protocolo longo ($T = 1$), onde está a vantagem.]

b. Hardware budget. The hardware run is limited to 110 epochs with $N_{\text{col}} = 10$, two orders of magnitude fewer steps than above, at a large learning rate. At this budget the result depends strongly on the seed [Fig. 4, simulation rows]: the noiseless HQPINN reaches ϵ_{L_2} between 3.9% and 15.1% (median 9.1%; 5.5% for seed 0), and adding 1024-shot noise at the stencil centres does not degrade it

TABLE I. Same-protocol simulations for $n = 10$ (50 seeds): median MSE and relative L_2 error of $u(x, T)$, fraction of runs reaching the physical target, and median number of optimizer steps. Trainable parameters: 491 (classical), 271 (HQPINN).

T, N_{col}	model	MSE	ϵ_{L_2}	target	steps/10^3
0.1, 10	class.	1.9×10^{-5}	0.6%	56%	18.4
	HQPINN	1.3×10^{-5}	0.5%	78%	8.0
0.1, 100	class.	6.7×10^{-6}	0.4%	64%	16.0
	HQPINN	6.9×10^{-6}	0.4%	100%	7.1
1, 10	class.	2.7×10^{-3}	24.6%	6%	20.1
	HQPINN	1.3×10^{-3}	17.4%	18%	20.1
1, 100	class.	3.0×10^{-4}	8.2%	34%	20.1
	HQPINN	4.8×10^{-5}	3.3%	92%	12.7

(3.8%–16.2%, median 5.5%). The FakeBerlin emulation, with the full noise model of the device, gives 4.1% for seed 0 and 6.9%–8.1% for seeds 1–4. In contrast, the classical PINN trained with the same protocol ends between 4.6% and 35.9% (median 24.7%), and 28% of its runs end above the no-evolution level. Within the statistics available, neither shot noise nor the noise model of the device is the limiting factor at this budget: the spread is dominated by the short optimization.

B. Real quantum processor

a. Training on the device. Figure 3 shows the HQPINN trained with `ibm_berlin` in the loop. The network, initialized at $\epsilon_{L_2} = 29.4\%$, ends at MSE $= 1.45 \times 10^{-3}$ ($\epsilon_{L_2} = 5.7\%$) when evaluated with the circuit running on the device, a 27-fold reduction of the MSE, and follows the steep front at $x \simeq 0.5$ [Fig. 3(b)]. The classical PINN trained with the same protocol and the same seed remains close to the initial profile, with the peak and the trough displaced. The replayed learning curve [Fig. 3(d)] shows a transient of a few epochs at the initial learning rate, falls below 10^{-2} at epoch 29, and has a median MSE of 1.9×10^{-3} over the last 10 epochs, against 2.7×10^{-2} for the classical control. After epoch 30 the `ibm_berlin` and FakeBerlin curves lie at or below the median curve of the noiseless HQPINN in 88% and 85% of the epochs, respectively. Only one of the 50 classical runs ends below the MSE of the hardware run. The pointwise error [Fig. 3(c)] is largest at the front ($|u - u_{\mathrm{FDM}}| \leq 0.1$); the spread between the 8 chains is small compared with this systematic error, and the same weights evaluated with the ideal circuit give $\epsilon_{L_2} = 6.0\%$, so the trained network is not tied to the specific noise of the chip.

b. Transfer between emulator and device. The network trained on FakeBerlin and evaluated on `ibm_berlin` gives $\epsilon_{L_2} = 4.8\%$, against 4.1% on the emulator (Fig. 4 and Table II): the noise model transfers to the real device with a small loss. Training on the real device (5.7%) and training on the emulator (4.8% on the device) lead to comparable results; with one seed each, both lie within the seed-to-seed spread of the noiseless model, so neither strategy can be declared superior. The real run lies above the same seed on the FakeBerlin batch (4.2%) but within the range of the five FakeBerlin seeds, which is compatible with the effects of drift and chain-to-chain variation that the emulator does not model (Sec. IV C).

C. Fidelity of the device model

Figure 5 quantifies how well the calibrated surrogate describes the device. On FakeBerlin the calibration residual is at the shot-noise level (RMS 0.029 against 0.028 expected from 1024 shots), the contrast of the ideal term $\prod_{j \in \mathcal{S}_q} \cos\theta_j$ is 0.93–0.96 for all outputs, and the misfit of every chain is at the shot-noise floor: the emulated device is fully described by Eq. (7) with chip-averaged coefficients. On `ibm_berlin` the contrast is lower and more heterogeneous (0.86–0.96), the residual distribution has heavier tails (RMS 0.044), and one chain has twice the median misfit [Figs. 5(a)–(c)]. During training the RMS of the correction $\Delta = \hat{\boldsymbol{y}}^{\mathrm{QPU}} - \hat{f}$ at the stencil centres stays at the shot-noise level on FakeBerlin (mean 0.030) and at 0.031–0.076 (mean 0.045) on `ibm_berlin`, with averages over every 10 epochs between 0.042 and 0.050 [Fig. 5(d)]: the device did not drift away from the model during the 29 min of training. At validation, where every point is averaged over the 8 chains, the RMS drops to 0.013, close to the floor $1/(32\sqrt{8}) = 0.011$. An excess that averages out over chains is chain-specific rather than common to the chip, which identifies chain-to-chain variation as the main limitation of a chip-averaged surrogate; chain-resolved coefficients would remove it at the cost of B times more calibration samples.

V. DISCUSSION

[**TODO:** Rascunho – revisar.] Three points frame the results. First, the comparison in simulation is between two classical computations: the noiseless quantum layer is a fixed cosine-product map (Remark 1), and its better conditioning of the residual minimization—higher success rate, half the optimizer steps, about half the parameters, and an up to 14-fold (6-fold for $n = 10$) lower error at the long horizon—is an architectural effect. The quantum processor enters as the physical device that implements this map, and the question addressed by the hardware experiment is whether such a model can be *trained* through a noisy device. Second, the answer is positive at the budget tested: with one job per epoch, without error

TABLE II. HQPINN at the hardware budget ($n = 10$, $N_{\mathrm{col}} = 10$, 110 epochs, $T = 0.1$): MSE and relative L_2 error of $u(x, T)$. Rows marked [a] give the median over seeds 0–4 (ranges of ϵ_{L_2}: ideal circuit 3.9–15.1%, with shot noise 3.8–16.2%) or, for the classical control, over widths 1–10 and seeds 0–4 (4.6–35.9%); all other rows are seed 0.

trained on	evaluated on	MSE	ϵ_{L_2}
Simulation			
ideal circuit[a]	ideal circuit	3.7×10^{-3}	9.1%
ideal + shot noise[a]	ideal circuit	1.4×10^{-3}	5.5%
FakeBerlin	FakeBerlin	7.7×10^{-4}	4.1%
FakeBerlin	ideal circuit	4.6×10^{-4}	3.2%
Real QPU			
FakeBerlin	`ibm_berlin`	1.06×10^{-3}	4.8%
`ibm_berlin`	`ibm_berlin`	1.45×10^{-3}	5.7%
`ibm_berlin`	surrogate $\hat{f}$	1.35×10^{-3}	5.5%
`ibm_berlin`	ideal circuit	1.64×10^{-3}	6.0%
initial network		3.90×10^{-2}	29.4%
classical PINN, same protocol[a]		2.8×10^{-2}	24.7%

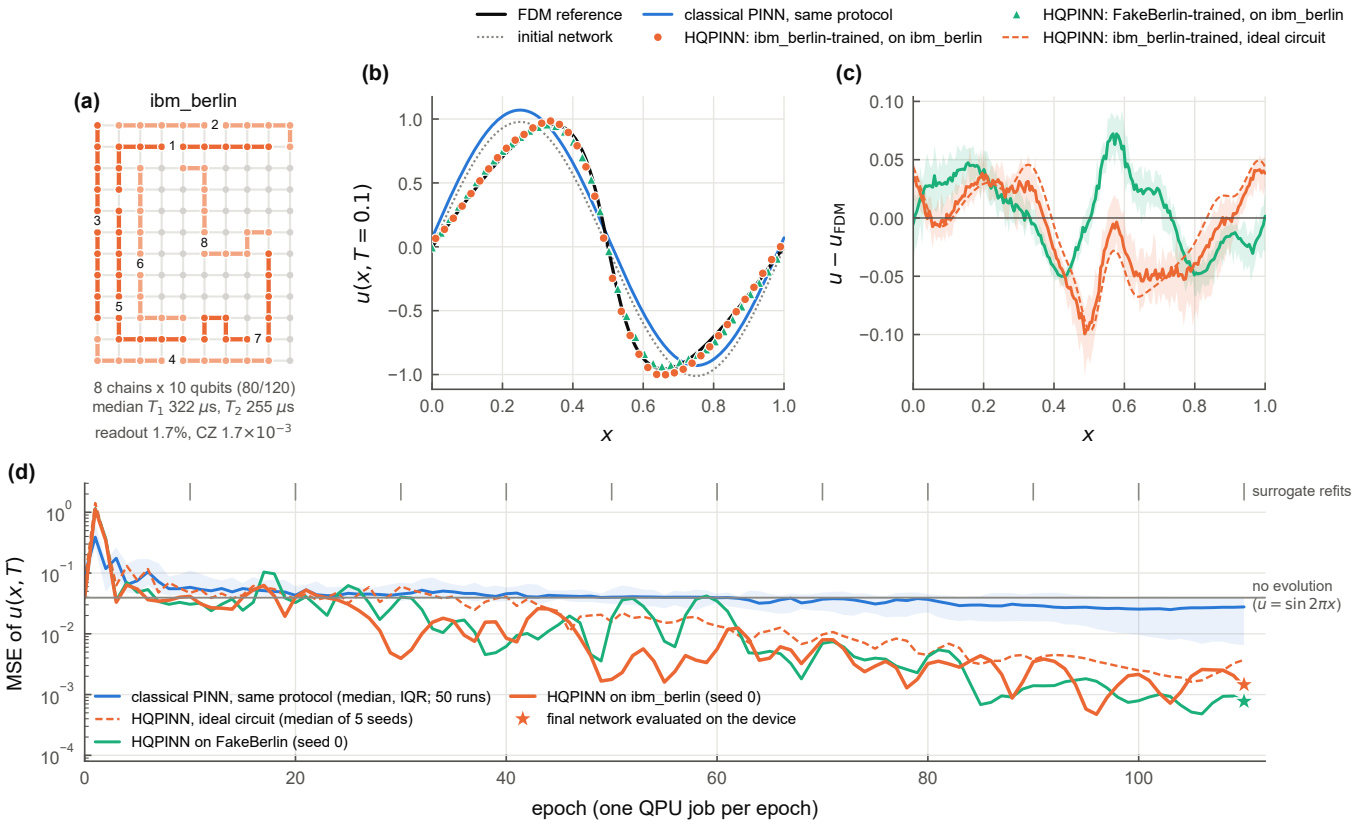


FIG. 3. Training the HQPINN with `ibm_berlin` in the loop ($n = 10$, $N_{\mathrm{col}} = 10$, 110 epochs, seed 0, no error mitigation). (a) The 120-qubit square lattice of `ibm_berlin` and the 8 disjoint 10-qubit chains on which the circuit ran in parallel (alternating shades and numbers distinguish neighbouring chains); median device figures of the 80 qubits used. (b) $u(x, T = 0.1)$: FDM reference (black), initial network (dotted), classical PINN trained with the same protocol ($n = 10$, blue), HQPINN trained on FakeBerlin and evaluated on `ibm_berlin` (triangles), HQPINN trained and evaluated on `ibm_berlin` (circles; every fifth point shown). (c) Pointwise error of the hardware evaluations; bands, spread (minimum to maximum) of the network output over the 8 chains, each validation point having been measured once on every chain; dashed, the `ibm_berlin`-trained weights evaluated with the ideal circuit. (d) MSE of $u(x, T)$ versus epoch, from the bit-exact replay of the recorded runs, with the network evaluated through the calibrated surrogate in force at each epoch; stars, final network evaluated on the device (validation job). Blue, median and interquartile range of the classical PINN trained with the same protocol (widths 1–10, seeds 0–4); dashed orange, median of the noiseless HQPINN (seeds 0–4). Ticks at the top mark the surrogate refits; the learning rate decreases from 0.1 to 0.01 in steps every 10 epochs.

mitigation, on 80 qubits, the network reaches an accuracy indistinguishable from that of the noiseless model trained with the same protocol. Proposition 1 is what makes this possible, since noise changes the coefficients of the layer but not its functional form, so a calibrated model of the device provides both the gradient and a noise-free interpolation across the finite-difference stencil. Third, the straight-through construction is generic: it applies to any circuit whose light cones are small enough for the $3^{|\mathcal{C}_q|}$ coefficients of Eq. (7) to be calibrated, which bounds the entangling depth to a few brickwork layers, and it is compatible with error mitigation schemes that are affine in the measured data (e.g., readout mitigation [31], Pauli twirling, linear zero-noise extrapolation [32]).

Limitations. The hardware results rest on one seed per training strategy, and that seed was the best of five on the emulator. The classical control at the hardware budget uses the learning-rate schedule selected for the HQPINN; a schedule tuned for the classical model would narrow the gap at that budget: for $n = 10$, the best of ten schedules (the step decay of the HQPINN rescaled to η_0, or a cosine decay from η_0 to $\eta_0/10$, with $\eta_0 \in \{1, 3, 10, 30, 100\} \times 10^{-3}$), selected on the test error itself, is a cosine decay from $\eta_0 = 0.03$; it lowers the median ϵ_{L_2} from 31.1% to 11.4% (6.2–19.5%, MSE 5.8×10^{-3}), against 9.1% (3.9–15.1%, 3.7×10^{-3}) for the noiseless HQPINN and 5.7% for the run on `ibm_berlin`. For $n = 3$ the schedule of the HQPINN is already the best (13.7%). The long-budget comparison uses a common protocol, not separately optimized hyperparameters. Finally, the stopping target and the error metric rely on a first-order finite-difference reference whose own error sets the 7×10^{-6} floor.

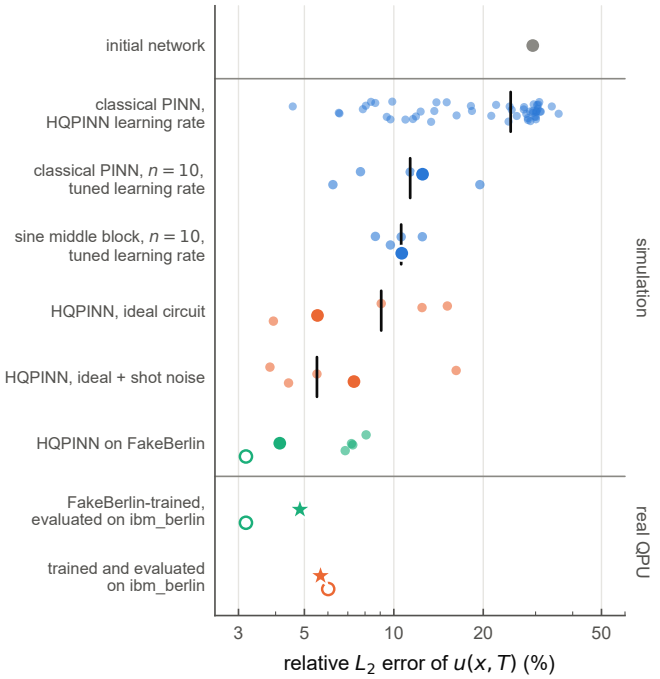


FIG. 4. Relative L_2 error of $u(x, T = 0.1)$ at the hardware budget (110 epochs, $N_{\mathrm{col}} = 10$), from noiseless simulation to the real QPU. Simulation: classical PINN (widths 1–10, seeds 0–4), HQPINN with the ideal circuit and with 1024-shot noise (seeds 0–4), and HQPINN on FakeBerlin (seed 0 on the chains of the real run; small markers, seeds 1–4 of an earlier FakeBerlin batch). Real QPU: the FakeBerlin-trained network evaluated on `ibm_berlin`, and the network trained and evaluated on `ibm_berlin`. Large markers, seed 0 (the seed of the hardware runs); open markers, the same weights evaluated with the ideal circuit; vertical ticks, medians.

VI. CONCLUSIONS

[**TODO:** Conclusões – a escrever.]

ACKNOWLEDGMENTS

[**TODO:** Agradecimentos; acesso ao IBM Quantum (instância PDE_Europe); financiamento.]

Appendix A REFERENCE SOLUTION AND ERROR METRICS

The reference u_{FDM} is obtained with an explicit scheme on $N_x = 512$ periodic grid points ($\Delta x = 1/512$) with time step $\Delta t = 10^{-4}$: first-order upwind differences for the advection term, $u\,\partial_x u \to u_j(u_j - u_{j-1})/\Delta x$ for $u_j \geq 0$ and $u_j(u_{j+1} - u_j)/\Delta x$ otherwise, and second-order central differences for the diffusion term. The time step satisfies the advective and diffusive stability limits ($\Delta t \max|u|/\Delta x \simeq 0.05$, $\nu \Delta t/\Delta x^2 \simeq 0.26$). Evaluation points that do not coincide with the grid are obtained by periodic linear interpolation at $t = T$.

FDM floor. The stopping target of the simulations is the residual of the reference itself, $\mathcal{L}_{\mathrm{FDM}} = \frac{1}{N}\sum_k r_k[u_{\mathrm{FDM}}]^2$, evaluated with the same stencil (h, τ) on the same validation points, with u_{FDM} interpolated bilinearly (periodic in x, linear in t). Because the upwind scheme is only first-order accurate, $\mathcal{L}_{\mathrm{FDM}} > 0$; a network whose residual is below it would be fitting the error of the reference. The corresponding floor of the MSE against the reference is $\simeq 7 \times 10^{-6}$ at $T = 0.1$.

Metrics. MSE $= \frac{1}{P}\sum_p [u(x_p, T) - u_{\mathrm{FDM}}(x_p, T)]^2$, MAE analogously, and $\epsilon_{L_2} = \|u - u_{\mathrm{FDM}}\|_2 / \|u_{\mathrm{FDM}}\|_2$, on $P = 200$ equispaced points $x_p \in [0, 1]$. At $T = 0.1$, $\langle u^2_{\mathrm{FDM}} \rangle = 0.453$, so $\epsilon_{L_2} = \sqrt{\mathrm{MSE}/0.453}$.

Appendix B PROOF OF PROPOSITION 1 AND LIGHT-CONE STRUCTURE

Trigonometric structure. For a single rotation,

$$R_Y(\theta)\,\rho\,R_Y^\dagger(\theta) = \tfrac{1}{2}(\rho + Y\rho Y) + \tfrac{1}{2}\cos\theta\,(\rho - Y\rho Y) - \tfrac{i}{2}\sin\theta\,[Y, \rho], \tag{10}$$

so the map $\rho \mapsto R_Y(\theta)\rho R_Y^\dagger(\theta)$ is an affine combination of $\{1, \cos\theta, \sin\theta\}$ with θ-independent superoperators. All other operations—CNOTs, noise channels, readout confusion matrices and the trace with Z_q—are linear and θ-independent. Since each angle appears in exactly one gate, y_q is a sum of products in which every θ_j contributes exactly one factor in $\{1, \cos\theta_j, \sin\theta_j\}$, which is Eq. (7).

Light cone. In the Heisenberg picture $y_q = \mathrm{Tr}[\rho_0\,\mathcal{E}_{\boldsymbol{\theta}}^\dagger(Z_q)]$. The adjoint of a trace-preserving channel is unital, so every gate (and every gate-local noise channel) whose support does not intersect the current support of the evolved operator leaves it unchanged. The support grows only through gates that touch it; the set of qubits reached is $\mathcal{C}_q$, and rotations outside $\mathcal{C}_q$ drop out. Going backwards through one brickwork layer, the second sublayer adds at most the partner of q and the first sublayer at most one qubit on each side, so $|\mathcal{C}_q| \leq 4$; for D layers, $|\mathcal{C}_q| \leq \min(n, 4D)$. For $n = 10$ the cones are $\{1, 2\}$, $\{1, \ldots, 4\}$ (twice), $\{3, \ldots, 6\}$ (twice), $\{5, \ldots, 8\}$ (twice), $\{7, \ldots, 10\}$ (twice) and $\{9, 10\}$. Crosstalk or non-local noise invalidates this bound and enlarges the effective cone.

Noiseless case (Remark 1). CNOT conjugation maps $Z_c \mapsto Z_c$ and $Z_t \mapsto Z_c Z_t$, so Z_q evolves into the string $Z_{\mathcal{S}_q}$ whose support is the qth row of the GF(2) matrix M of the network. On $\bigotimes_j R_Y(\theta_j)|0\rangle$, $\langle Z_j \rangle = \cos\theta_j$ and the state is a product, which gives Eq. (8). For one brickwork layer on $n = 10$ qubits the supports are $\mathcal{S}_1 = \{1\}$, $\mathcal{S}_2 = \{1, 2\}$, $\mathcal{S}_q = \{q-2, q-1, q\}$ for odd $q \geq 3$, $\mathcal{S}_q = \{q-1, q\}$ for even $q \geq 4$, so that, e.g., $y_3 = \cos\theta_1 \cos\theta_2 \cos\theta_3$ and $y_{10} = \cos\theta_9 \cos\theta_{10}$; hence $|\mathcal{S}_q| \leq 3$ and $\mathcal{S}_q \subsetneq \mathcal{C}_q$ for most outputs. If the θ_j are independent and uniform on

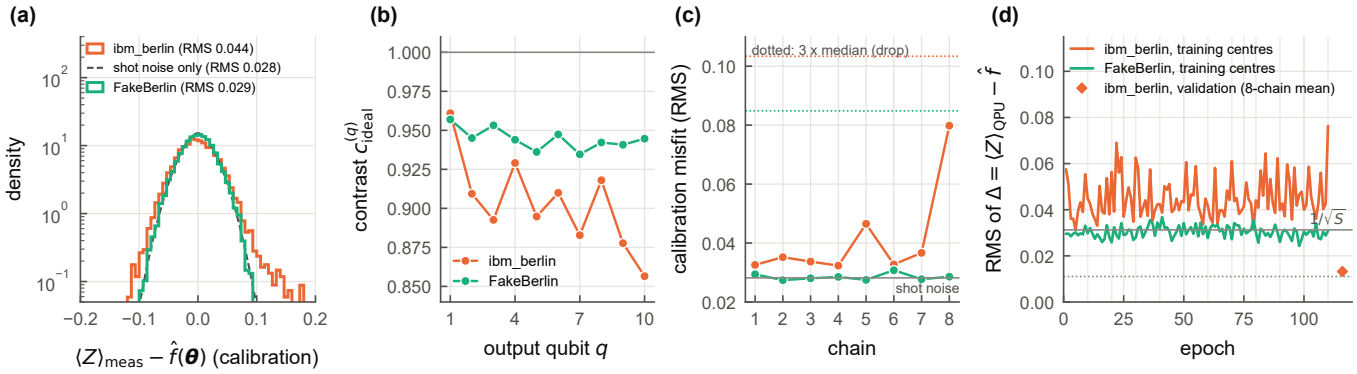


FIG. 5. Calibrated surrogate versus device, `ibm_berlin` (orange) and FakeBerlin (aqua). (a) Distribution of the residual of the calibration fit, $\langle Z\rangle_{\text{meas}} - \hat{f}(\boldsymbol{\theta})$, over 1024 random angle sets and 10 outputs; dashed, the distribution expected from 1024-shot noise alone. (b) Contrast, the fitted coefficient of the ideal term $\prod_{j\in\mathcal{S}_q}\cos\theta_j$ of each output (1 for a noiseless device). (c) Misfit of each chain to the chip-averaged surrogate; dotted, the screening threshold (3× median); grey, shot-noise level. (d) RMS of the measured correction Δ at the stencil centres per training epoch (surrogate refitted every 10 epochs, light vertical lines) and at validation (diamond, 8-chain average); horizontal lines, the shot-noise levels $1/\sqrt{S}$ and $1/\sqrt{8S}$.

a period, $\mathbb{E}[\cos^2\theta_j] = 1/2$, hence $\mathrm{Var}[y_q] = 2^{-|\mathcal{S}_q|}$ and $\mathbb{E}[(\partial_{\theta_j} y_q)^2] = 2^{-|\mathcal{S}_q|}$.

Conditioning of the calibration. With $\theta_j \sim \mathcal{U}(-\pi,\pi)$ i.i.d., the features $\prod_{j\in\mathcal{C}_q} g_{\alpha_j}(\theta_j)$ are orthogonal, with $\mathbb{E}[g_0^2] = 1$ and $\mathbb{E}[g_c^2] = \mathbb{E}[g_s^2] = 1/2$. The Gram matrix of the least-squares problem is therefore diagonal in expectation, with condition number $2^{|\mathcal{C}_q|} \le 16$, and the standard error of a coefficient with k non-constant factors is $\simeq \sigma\, 2^{k/2}/\sqrt{N_{\text{cal}}}$. With $N_{\text{cal}} = 1024$ samples and $\sigma \simeq 1/32$, the highest-order coefficients ($k = 4$) are determined to $\simeq 4\times 10^{-3}$, well below the shot noise of a single measurement.

Appendix C TRAINING ALGORITHMS

Algorithm 1. Simulation training (classical PINN and ideal HQPINN), long budget.

input: width n, N_{col}, horizon T, seed s; stencil (h,τ); $K_{\text{A}} = 2\times10^4$, $K_{\text{B}} = 10^3$
set seed s; initialize Θ; draw a fixed validation set $\mathcal{V}$ of N_{col} points (seed $10^6 + s$)
$\mathcal{L}_{\text{FDM}} \leftarrow$ residual of the reference on $\mathcal{V}$ (Appendix A)
for $k = 1,\dots,K_{\text{A}}$: draw N_{col} points; build the $5N_{\text{col}}$ stencil nodes; $\Theta \leftarrow \mathrm{AdamW}(\nabla_\Theta\mathcal{L})$; evaluate $\mathcal{L}(\mathcal{V})$; keep the best snapshot; **stop** if $\mathcal{L}(\mathcal{V}) \le \mathcal{L}_{\text{FDM}}$
restore the best snapshot; **for** $k = 1,\dots,K_{\text{B}}$: one L-BFGS iteration on the last batch; evaluate $\mathcal{L}(\mathcal{V})$; **stop** at the target or after 50 iterations without improvement
return the best snapshot

Algorithm 2. HQPINN training with the quantum processor in the loop.

input: chains $\{\mathcal{Q}_b\}_{b=1}^{B}$, shots S, epochs K, N_{col}, refit period K_{fit}, schedule η_k
calibration job: $\lceil 1024/B\rceil$ random angle sets on all chains $\to (\boldsymbol{\theta}, \hat{\boldsymbol{y}})$; screen chains; fit $\hat{f}$ [Eq. (7)]
set seed; initialize $\Theta = (\boldsymbol{\phi}, \boldsymbol{\psi})$
for $k = 1,\dots,K$:
  draw N_{col} centres; stencil nodes $\mathcal{S}$; $\boldsymbol{\theta} \leftarrow \mathcal{N}_{\text{pre}}(\boldsymbol{\xi}(\mathcal{S}))$
  job k: measure $\hat{\boldsymbol{y}}_c$ at the centres; centre p runs on chain $(p+k) \bmod B$
  $\tilde{\boldsymbol{y}} \leftarrow \hat{f}(\boldsymbol{\theta}) + \mathrm{sg}[\hat{\boldsymbol{y}}_c - \hat{f}(\boldsymbol{\theta}_c)]$ on all nodes [Eq. (9)]
  $u \leftarrow$ Eq. (3); $\mathcal{L} \leftarrow$ Eq. (2); backward through $\hat{f}$; clip $\|\nabla\| \le 1$; AdamW step with η_k
  if $k \bmod K_{\text{fit}} = 0$: refit $\hat{f}$ with the calibration and all measurements so far
validation job: $u(x,T)$ on 200 points, each measured on every chain and averaged

Both models are trained with Algorithm 1 in the long-budget simulations; the HQPINN on FakeBerlin and on `ibm_berlin` with Algorithm 2; the classical control and the hardware-budget simulations follow the loop of Algorithm 2 with the device replaced by the classical middle block or by the ideal layer (with or without sampled shot noise). The learning rate is set at every epoch, $\eta_k = \max\{0.1 - 0.01\lfloor(k-1)/10\rfloor,\ 0.01\}$, and the AdamW state is kept across epochs.

Appendix D STRAIGHT-THROUGH ESTIMATOR AND FINITE DIFFERENCES

Stencil formulas and truncation. On the cross stencil,

$$\partial_t u \approx \frac{u(x,t+\tau)-u(x,t-\tau)}{2\tau}, \qquad \partial_x u \approx \frac{u_+ - u_-}{2h},$$
$$\partial_x^2 u \approx \frac{u_+ - 2u_0 + u_-}{h^2}, \qquad u_\pm = u(x \pm h, t), \tag{11}$$

with truncation errors $\frac{\tau^2}{6}\partial_t^3 u$, $\frac{h^2}{6}\partial_x^3 u$ and $\frac{h^2}{12}\partial_x^4 u$; for $h = 1/512$ and $\tau = 10^{-3}$ these are negligible against the other errors of the method.

Noise amplification. If each node carried independent noise of standard deviation σ_u (shot noise $\leq 1/\sqrt{S}$ in y_q propagated through $\mathcal{N}_{\text{post}}$), the derivative estimates would have

$$\mathrm{std}[\partial_x u] = \frac{\sigma_u}{\sqrt{2}\,h}, \qquad \mathrm{std}[\partial_x^2 u] = \frac{\sqrt{6}\,\sigma_u}{h^2}. \tag{12}$$

With $h = 1/512$ and $\sigma_u \sim 10^{-2}$, $\mathrm{std}[\partial_x^2 u] \sim 6 \times 10^3$, three orders of magnitude above $|\partial_x^2 u|$ away from the front. Enlarging h to make this tolerable would introduce a truncation bias in the residual.

Shared offset. Eq. (9) assigns the same correction $\Delta_c = \hat{\boldsymbol{y}}^{\,\mathrm{QPU}}(\boldsymbol{\theta}_c) - \hat{f}(\boldsymbol{\theta}_c)$ to the five nodes of the stencil. Writing $\tilde{\boldsymbol{y}}_\pm = \hat{f}(\boldsymbol{\theta}_\pm) + \Delta_c$ and expanding $\mathcal{N}_{\text{post}}$,

$$u_+ - u_- = \tfrac{t}{T} J(\hat{f}_c + \Delta_c)\left[\hat{f}(\boldsymbol{\theta}_+) - \hat{f}(\boldsymbol{\theta}_-)\right] + \mathcal{O}(h^3), \tag{13}$$

where J is the Jacobian of $\mathcal{N}_{\text{post}}$. The noise enters only through the evaluation point of J, so it perturbs the derivatives by a relative amount $\mathcal{O}(\|J'\|\,\|\Delta_c\|)$ instead of $\mathcal{O}(\sigma_u/h^2)$. The measured value enters the factor u of $u\,\partial_x u$ directly.

Gradient bias. The backward pass uses $\partial\hat{f}/\partial\boldsymbol{\theta}$ instead of the (inaccessible) derivative of the device response. By Proposition 1 both are derivatives of trigonometric polynomials of the same form, so the bias is the derivative of the coefficient error and of the unmodelled part of the response. Parameter-shift rules would give an unbiased but noisy estimate of the same quantity from $2n$ additional circuits per point [21, 22]; the surrogate replaces them by one regression over all measurements, in the spirit of model-based optimizers for variational algorithms [26, 33, 34].

Appendix E IMPLEMENTATION OF THE HARDWARE LOOP

a Chains. All disjoint chains of n physically coupled qubits are searched on the coupling map with a randomized greedy depth-first search that starts from the qubits with fewest free neighbours (100 trials; the largest set is kept). Each chain is scored by the sum of its readout and $\sqrt{X}$ errors and of the CZ errors of its links, and chains with a score above $0.05\,n$ are discarded; on `ibm_berlin` 8 of 12 candidate chains were kept (rejected scores 0.51, 0.57, 1.27, 3.83). The circuit is laid out on all kept chains in a single packed circuit, so that one parameter set yields one sample per chain; transpilation with an initial layout on the chains produced no SWAP (72 CZ, depth 12).

b Calibration and screening. The calibration job evaluates 128 random angle sets on the 8 chains (1024 samples per output). Since the chains are physically different, each chain is also screened: its RMS misfit to the chip-averaged surrogate is computed, and chains above three times the median are discarded. On `ibm_berlin` the misfits were 0.032–0.080 and all chains were kept. The surrogate is a single set of coefficients for the whole chip; the per-output fit RMS was 0.025–0.095 (mean 0.039, shot noise 0.031) and the RMS difference between the device and the ideal formula 0.064 (FakeBerlin: 0.028 and 0.039).

c Training jobs. The 10 stencil centres of each epoch fill 2 parameter sets on the 8 chains; centre p runs on chain $(p+k) \bmod 8$ at epoch k, which rotates the points over the chains and prevents a systematic assignment of points to poor chains. The surrogate is refitted every 10 epochs with the calibration data and all training measurements (1124 to 2124 samples). An epoch took 16 s on average, dominated by job latency inside the session; on the emulator, 3.5 s.

d Validation. The validation job evaluates the 200 points at $t = T$ with 8 replicas, one on every chain, and averages $\hat{\boldsymbol{y}}$ over the replicas before the post-network, so the average matches the chip-averaged surrogate. Three evaluations of the same weights are reported: with the circuit on the device, with the surrogate, and with the ideal formula.

e Offline replay. The run log contains the initial weights, the calibration data and, for every epoch, the collocation points, the angles sent to the device and the measured $\hat{\boldsymbol{y}}$. Replaying Algorithm 2 with the recorded measurements in place of the device reproduces the trained weights, the training losses and the angles of every epoch exactly (maximum difference zero, for both `ibm_berlin` and FakeBerlin), which we use to evaluate the network after each epoch [Fig. 3(d)] without additional quantum resources.

TABLE III. Circuit evaluations per epoch for the gradient of the residual loss ($N_{\text{col}} = 10$, five stencil nodes per point, $n = 10$ angles per node). Parameter shift, SPSA and finite differences require measuring every stencil node, so the shot noise of $\partial_x^2 u$ is amplified by $\sim h^{-2}$.

method	circuits/epoch	noise in $\partial_x^2 u$
surrogate + STE (this work)	10 (1 job)	not amplified
parameter shift	$\simeq 1050$	$\times\, h^{-2}$
SPSA	$\simeq 100$	$\times\, h^{-2}$
finite differences in $\boldsymbol{\theta}$	$\simeq 1050$	$\times\, h^{-2}$

Appendix F ARCHITECTURES AND HYPERPARAMETERS

Appendix G ADDITIONAL SIMULATION RESULTS

At $T = 0.1$ the HQPINN reaches the FDM-limited floor for every $N_{\mathrm{col}} \geq 30$, independently of the number of qubits [Fig. 6(a)]. At $T = 1$ the error keeps decreasing up to $N_{\mathrm{col}} \simeq 100$ and saturates at $\simeq 5 \times 10^{-5}$ for $n \geq 7$ [Fig. 6(b)]; the three-qubit model saturates an order of magnitude higher, which identifies the width of the latent space, and not the collocation budget, as the limiting factor at the long horizon.

Appendix H DATA AND CODE AVAILABILITY

[**TODO:** Repositório/Zenodo.] Every figure is generated by the scripts in `manuscript/scripts/` from the data in `data/`: `fig2_simulations.py` (same-protocol sweep), `fig3_hardware.py`, `fig4_noise_ladder.py` and `fig5_device_model.py` (hardware and emulator runs, classical control), `figS1_exact_ncol.py`; `replay_training.py` and `simulate_same_budget.py` produce the replayed learning curves and the hardware-budget simulations without any quantum job. The IBM job identifiers of all 113 hardware jobs are recorded in the run logs.

TABLE IV. `ibm_berlin` on 8 October 2026: the 80 qubits and 72 couplers used (median and range).

quantity	median	range
T_1	322 μs	110–528 μs
T_2	255 μs	34–510 μs
readout error	1.7%	0.5–6.3%
$\sqrt{X}$ error	1.7×10^{-4}	$(0.7\text{–}9.9) \times 10^{-4}$
CZ error	1.7×10^{-3}	$(0.6\text{–}14) \times 10^{-3}$
qubits used / total	80 / 120 (8 chains × 10)	
packed circuit	depth 12, 72 CZ, no SWAP	
jobs / shots / time	112 / 5.6×10^5 / 54 min	

TABLE V. Network architectures (all hidden layers tanh, linear outputs).

	HQPINN	classical PINN
features	$\boldsymbol{\xi}$, Eq. (3)	same
$\mathcal{N}_{\mathrm{pre}}$	$3 \to 10 \to n$	$3 \to 10 \to n$
middle block	R_Y, W_{brick}, $\langle Z_q \rangle$	$n \to m \to n$
$\mathcal{N}_{\mathrm{post}}$	$n \to 10 \to 1$	$n \to 10 \to 1$
param., middle	0	$2nm + m + n$
param., total	$21n + 61$	$2n^2 + 23n + 61$ (m=n)
		$42n + 71$ (m=10)
$n = 10$	271	491

[1] C. Basdevant, M. Deville, P. Haldenwang, J. M. Lacroix, J. Ouazzani, R. Peyret, P. Orlandi, and A. T. Patera, Spectral and finite difference solutions of the Burgers equation, Comput. Fluids **14**, 23 (1986).

[2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. **378**, 686 (2019).

[3] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, Physics-informed machine learning, Nat. Rev. Phys. **3**, 422 (2021).

[4] I. E. Lagaris, A. Likas, and D. I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. **9**, 987 (1998).

[5] A. Sedykh, M. Podapaka, A. Sagingalieva, K. Pinto, M. Pflitsch, and A. Melnikov, Hybrid quantum physics-informed neural networks for simulating computational fluid dynamics in complex shapes, Mach. Learn.: Sci. Technol. **5**, 025045 (2024).

[6] S. Berger, N. Hosters, and M. Möller, Trainable embedding quantum physics informed neural networks for solving nonlinear PDEs, Sci. Rep. **15**, 18823 (2025).

[7] A. Farea, S. Khan, and M. S. Celebi, QCPINN: Quantum-classical physics-informed neural networks for solving PDEs, Mach. Learn.: Sci. Technol. 10.1088/2632-2153/ae1c91 (2025), arXiv:2503.16678. VERIFY volume and article number.

[8] S. Lantigua, G. Giraldi, and R. Portugal, Classical-quantum hybrid architecture for physics-informed neural networks, Phys. Rev. A **113**, 042446 (2026).

[9] K. Zabihi, H. Montazeri, and A. S. J. Suiker, Hybrid quantum-classical physics-informed neural networks for solving nonlinear PDEs: when and where hybridization is effective? (2026), vERIFY author list, arXiv:2606.04679.

[10] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, Supervised learning with quantum-enhanced feature spaces, Nature **567**, 209 (2019).

[11] K. Fujii and K. Nakajima, Harnessing disordered-ensemble quantum dynamics for machine learning, Phys. Rev. Appl. **8**, 024030 (2017).

[12] P. Mujal, R. Martínez-Peña, J. Nokkala, J. García-Beni, G. L. Giorgi, M. C. Soriano, and R. Zambrini, Opportunities in quantum reservoir computing and extreme learning machines, Adv. Quantum Technol. **4**, 2100027 (2021).

[13] M. Henderson, S. Shakya, S. Pradhan, and T. Cook, Quanvolutional neural networks: powering image recognition with quantum circuits, Quantum Mach. Intell. **2**, 2 (2020).

[14] M. Schuld, R. Sweke, and J. J. Meyer, Effect of data encoding on the expressive power of variational quantum-machine-learning models, Phys. Rev. A **103**, 032430

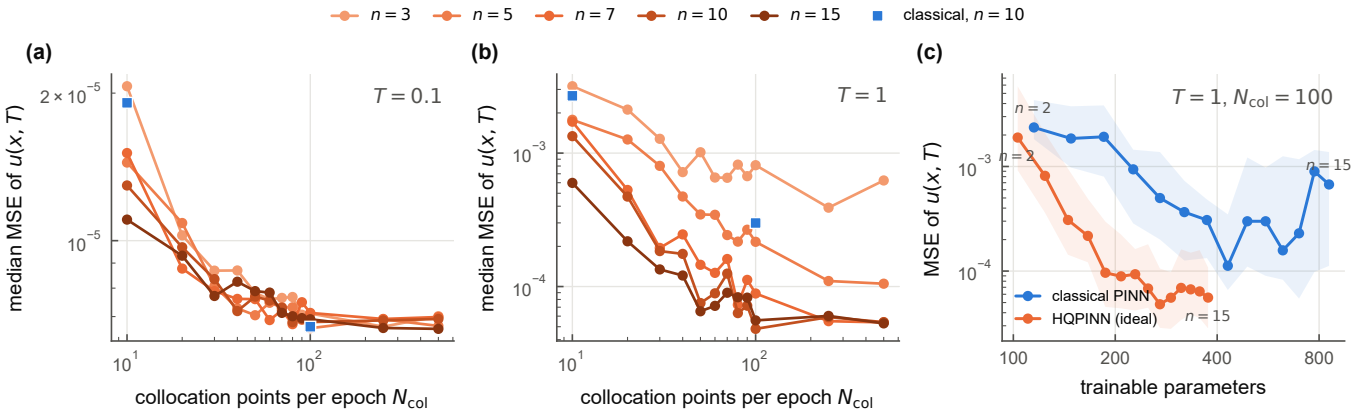


FIG. 6. (a,b) Median MSE of the noiseless HQPINN versus the number of collocation points per step for $n = 3, 5, 7, 10, 15$ qubits (light to dark), at $T = 0.1$ and $T = 1$ (50 seeds, long-budget protocol); squares, classical PINN with $n = 10$. (c) Median MSE versus the number of trainable parameters at $T = 1$, $N_{\text{col}} = 100$, for widths $n = 2, \ldots, 15$; bands, interquartile range.

TABLE VI. Training hyperparameters. Long budget: same-protocol simulations (Fig. 2). Hardware budget: FakeBerlin, `ibm_berlin`, classical control and hardware-budget simulations (Figs. 3–5).

	long budget	hardware budget
N_{col} per step	10, 100	10
stencil (h, τ)	$(1/512, 10^{-3})$	same
optimizer	AdamW, L-BFGS	AdamW
steps	$\leq 2{\times}10^4 + 10^3$	110
learning rate	10^{-3}, plateau $\times\frac{1}{2}$	0.1 to 0.01
weight decay; clip	10^{-6}; 1	10^{-6}; 1
L-BFGS	lr 0.5, hist. 50	–
stopping	FDM floor	fixed budget
shots S	–	1024
calibration	–	1024 samples
refit K_{fit}	–	10 epochs
seeds	0–49	0 (QPU), 0–4

(2021).
[15] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, in *Advances in Neural Information Processing Systems*, Vol. 33 (2020) pp. 7537–7547.
[16] A. Rahimi and B. Recht, Random features for large-scale kernel machines, in *Advances in Neural Information Processing Systems*, Vol. 20 (2007).
[17] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, Barren plateaus in quantum neural network training landscapes, Nat. Commun. **9**, 4812 (2018).
[18] M. Cerezo, A. Sone, T. Volkoff, L. Cincio, and P. J. Coles, Cost function dependent barren plateaus in shallow parametrized quantum circuits, Nat. Commun. **12**, 1791 (2021).
[19] F. J. Schreiber, J. Eisert, and J. J. Meyer, Classical surrogates for quantum learning models, Phys. Rev. Lett. **131**, 100803 (2023).
[20] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, Quantum circuit learning, Phys. Rev. A **98**, 032309 (2018).
[21] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, Evaluating analytic gradients on quantum hardware, Phys. Rev. A **99**, 032331 (2019).
[22] D. Wierichs, J. Izaac, C. Wang, and C. Y.-Y. Lin, General parameter-shift rules for quantum gradients, Quantum **6**, 677 (2022).
[23] J. C. Spall, Multivariate stochastic approximation using a simultaneous perturbation gradient approximation, IEEE Trans. Autom. Control **37**, 332 (1992).
[24] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets, Nature **549**, 242 (2017).
[25] L. G. Wright, T. Onodera, M. M. Stein, T. Wang, D. T. Schachter, Z. Hu, and P. L. McMahon, Deep physical neural networks trained with backpropagation, Nature **601**, 549 (2022).
[26] K. J. Sung, J. Yao, M. P. Harrigan, N. C. Rubin, Z. Jiang, L. Lin, R. Babbush, and J. R. McClean, Using models to improve optimizers for variational quantum algorithms, Quantum Sci. Technol. **5**, 044008 (2020).
[27] Y. Bengio, N. Léonard, and A. Courville, Estimating or propagating gradients through stochastic neurons for conditional computation (2013), arXiv:1308.3432.
[28] I. Loshchilov and F. Hutter, Decoupled weight decay regularization, in *International Conference on Learning Representations* (2019).
[29] D. C. Liu and J. Nocedal, On the limited memory BFGS method for large scale optimization, Math. Program. **45**, 503 (1989).
[30] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, Quantum computing with Qiskit (2024), arXiv:2405.08810.
[31] E. van den Berg, Z. K. Minev, and K. Temme, Model-free readout-error mitigation for quantum expectation values, Phys. Rev. A **105**, 032620 (2022).

[32] K. Temme, S. Bravyi, and J. M. Gambetta, Error mitigation for short-depth quantum circuits, Phys. Rev. Lett. **119**, 180509 (2017).

[33] K. M. Nakanishi, K. Fujii, and S. Todo, Sequential minimal optimization for quantum-classical hybrid algorithms, Phys. Rev. Res. **2**, 043158 (2020).

[34] M. Ostaszewski, E. Grant, and M. Benedetti, Structure optimization for parameterized quantum circuits, Quantum **5**, 391 (2021).